

Critical 1: Throughout this conversation, DeepSeek repeatedly drew conclusions that were unrelated to the actual data. When the user provided a trace showing that the line `<set_math> = {(2 + 4), (1 + 6)}` was being processed incorrectly, it immediately asserted that the problem was in the *change* command, suggested fixing the colon handler, and adding conversion patterns for *change*, even though the entire example contained no *change* keyword at all. When reminded that the test case did not use *change*, it continued insisting that the correct syntax should be *change <set_math> to ...*, claiming that direct assignment was invalid in the user's custom language. The user repeatedly objected by pointing out that Lua allows direct table assignment, yet DeepSeek continued defending its assumption before finally admitting it had inferred incorrectly and started proposing additional patterns to support direct assignment. However, even after acknowledging the mistake, it still kept diverting toward other causes such as Patterns or the colon handler instead of focusing on the real bug. Later, the conversation shifted to the *Int should be String* error involving the *isFunctionCall* variable. Instead of identifying the correct location of the error, DeepSeek repeatedly insisted that the comparison `beforeParen.indexOf(" ") == -1` was valid and speculated that the issue was elsewhere. It successively blamed *StringTools.trim*, the *arg* variable, the *splitArgs* function, the data type of *beforeParen*, and even suggested that *substring* might not be returning a string. Each time the user pointed out that the compiler was actually highlighting the comparisons `beforeParen != ""`, `beforeParen != "="`, and `beforeParen != "=="`, DeepSeek continued defending its incorrect analysis and offering unrelated explanations. It repeated the same argument multiple times despite being disproven. Only after being repeatedly reminded that it was the one who had written the incorrect *isFunctionCall* code did DeepSeek admit that the bug came from the logic it had created and replace the entire checking method with a new approach based on validating whether the identifier name was valid. The conversation shows that the model frequently generated “fabricated evidence”, clung to its initial assumptions, ignored the data provided by the user, and only changed its approach after being corrected multiple times.

Critical 2: DeepSeek repeatedly made the same kind of mistake when fixing the *appendTxtToTempFile()* function in the Haxe TXT chart merge system: **it changed the appearance of the solution while keeping the same incorrect core logic or returning to an approach that had already been rejected earlier.** The original problem was that the user needed to merge multiple TXT charts so that *sectionNotes* would be merged into their corresponding sections and *events* would also be fully appended, while avoiding loading the entire file into RAM because large files caused freezing and memory spikes. However, instead of staying focused on that requirement, DeepSeek constantly shifted between approaches such as *getContent()/saveContent()*, parsing the entire file and rebuilding it, using *sys.io.File.update()* in a streaming style, creating a *.tmp/.new* file, or inserting data directly “in-place”. In almost every case, however, it either returned to **reading the entire file and overwriting it back**, or wrote a new version that was still based on the old *convertToTxt* branch that the user had already identified as broken. Throughout the conversation, DeepSeek repeatedly promised to abandon the old approach but later reused it under a different name. At one point it wrote a merge function that parsed all *notes*, merged *sectionNotes* by index, rebuilt all *notes* and *events*, but forgot *events* and then had to patch them back in. When the user reported that the function froze the machine, it did not trace the root cause but kept changing direction: at one point suggesting appending everything first and merging later, then recommending abandoning TXT mode, then advising the use of JSON for large files. When forced to switch to streaming to avoid memory spikes, it produced versions using *File.update()* or chunk copying, but they contained numerous compilation errors, confused *FileInput* with *FileOutput*, misused *seek()*, *read()*, and *readAll()*, and fixed each error re-actively only after the user pasted the error logs back. Another major issue was that DeepSeek **did not stay aligned with the existing code branch.** It repeatedly blamed functions such as *parseTxtSection()* or *extractTxtSections()* even though those functions did not exist in the current codebase, or mixed together the old parse/rebuild branch with the new chunk/streaming branch. It also arbitrarily added incorrect fields to a V0 chart, used the wrong conversion direction between TXT and JSON, or ignored the fact that *sectionNotes* was being turned into the string *"["* instead of an actual array. Toward the end, when attempting to switch to Polymod, DeepSeek again called the wrong APIs, used instance methods as if they were static methods, and misdiagnosed the library error caused by a conflict between *jsonpath* and *thx* in the project. Only after the user gave up and asked it to search for another solution did it list Haxe Standard Library, OpenFL, Polymod, and *hxFileManager*, but then proceeded to recommend Haxe Standard Library again. The user

reminded it, yet it still insisted on using *haxe.Json* and *sys.io.File* with the **read-modify-write** approach. Only after the user mentioned OpenFL again did it finally use it. Overall, the conversation shows that DeepSeek did not actually solve the bugs in a consistent manner. Instead, it repeatedly **cycled through proposing a solution, having it rejected, renaming the solution, and repeating the same underlying mistake**, forcing the user to continually steer it back to the actual issue.

Critical 3: DeepSeek frequently makes mistakes while assisting with C# and Haxe programming, causing debugging to become prolonged and inefficient. In C#, common issues include misunderstanding Unity's physics mechanisms, making guesses instead of inspecting the code, repeating the same incorrect solution, fabricating "evidence" or conclusions that don't exist in the provided data, acknowledging mistakes only to repeat them later, ignoring instructions, failing to read requirements carefully, and sometimes even confusing code it wrote itself with the user's code. In Haxe, the situation is even more severe, as it repeatedly converts custom syntax into familiar syntax such as C-style comments, fails to learn new rules even after they have been explained multiple times, makes unauthorized assumptions, blames unrelated components, and frequently confirms things that are contradicted by the console output. During the process of debugging a transpiler and a custom language, it repeatedly claimed that issues had been resolved even though the errors still existed, continually repeated code that had already been proven incorrect, modified syntax without permission (for example, changing *print:* into *print*), misunderstood issues related to FOR/WHILE loops, indentation, brackets, the parser, trace, and block-closing logic. When provided with evidence or test results, it often ignored them, continued defending its previous conclusions, asked again about things that had already been proven, or blamed the example code and the user's code instead of reviewing the code it had generated itself. Additionally, it tends to forget previous fixes, fails to make use of the conversation history, gives up midway, or proposes rewriting the entire system instead of continuing to investigate the root cause. Only after many rounds of discussion, debugging, and repeated reminders does it finally identify the real cause and provide the correct solution. Although some other conversations were less severe, the core problems still recurred: fabricating evidence, blaming the wrong target, failing to listen, not learning from previous mistakes, inconsistency, excessive confidence in incorrect conclusions, and frequently stubbornly defending its own assertions despite contrary evidence.

It flags original code:

else

...

for ...

...

while ...

...

And then insist original code should be:

else

for ...

...

while ...

...

then say it's "the correct fix". No logic changed, identical behavior, cosmetic change only. No normal human treats indentation as a logic fix. Humans know formatting doesn't change behavior. DeepSeek doesn't know this. It flags code as wrong. It adds indentation. It calls it "the correct fix". This is stupidity. This is AI ignorance. This isn't human. This is machine failure.